Wantam Holdings
Banking Management System

A Secure, Scalable, and Cross-Platform Digital Banking Platform

Course: ACS311

Submission Date: August 2026

GitHub Repository: https://github.com/rayv7/wantamHoldings

2. Group Members

#	Name	Student ID	Role
1	Raymond Athanas	24-2578	Project Lead / Full-Stack Developer
2	Obutu Enoch	22-0950	Backend Developer
3	Humphery Kusimba	23-2552	Frontend Developer
4	Julia Selvine	23-2759	UI/UX Designer
5	Jakes Murila	24-2975	Backend Developer

3. Introduction

The Wantam Holdings Banking Management System is a modern, secure, and cross-platform digital banking application designed to bridge the gap between traditional banking services and the growing demand for accessible, real-time financial management. Built using Flutter for the frontend and NestJS for the backend, the system provides customers with a full suite of banking capabilities while equipping administrators with powerful tools to manage accounts, transactions, users, and financial reporting.

The application follows a client-server architecture, where the Flutter-based frontend communicates with a RESTful API backend through secure HTTPS connections. All banking data is stored in a PostgreSQL relational database accessed via the Prisma ORM, ensuring data integrity, security, and transactional consistency. The system implements industry-standard security practices including JWT-based authentication, BCrypt password hashing, and Role-Based Access Control (RBAC) to protect sensitive financial information.

The system supports multiple deployment platforms — including mobile (Android and iOS), web, and desktop — through Flutter's cross-platform capabilities. This ensures that customers can access their banking services from any device without sacrificing user experience or security. Additionally, the system includes USSD banking support for users without internet connectivity, making banking services accessible to a broader population.

Key features of the system include customer registration and authentication, account management, deposits and withdrawals, fund transfers between accounts, transaction history with search and filtering capabilities, loan management, and an administrative dashboard for monitoring and managing the entire banking ecosystem.

4. Problem Statement

Many small to medium financial institutions face significant challenges in delivering modern digital banking services to their customers. The key problems that this project addresses include:

- **Limited Accessibility:** Traditional banking services require customers to visit physical branches during operating hours, which is inconvenient and limits access to banking services, particularly in rural or underserved areas.
- **Fragmented Systems:** Many institutions rely on disparate, unintegrated systems for customer management, account operations, and transaction processing, leading to inefficiencies, data inconsistencies, and increased operational costs.
- **Security Concerns:** Financial systems are prime targets for cyberattacks. Without robust authentication mechanisms, encryption, and access control, customer funds and sensitive data are at risk.
- **Lack of Real-Time Visibility:** Customers and administrators alike need real-time access to account balances, transaction histories, and financial reports to make informed decisions.
- **Cross-Platform Gaps:** Many existing solutions are limited to a single platform (e.g., web-only or mobile-only), failing to provide a consistent experience across devices.
- **Connectivity Barriers:** Not all customers have reliable internet access. A digital banking solution must accommodate offline-capable or USSD-based alternatives to remain inclusive.

The Wantam Holdings Banking Management System aims to solve these problems by delivering a secure, unified, cross-platform banking platform that is accessible from mobile devices, web browsers, and even feature phones via USSD.

5. Objectives

5.1 General Objective

To design and develop a secure, scalable, and cross-platform Banking Management System that enables customers to perform digital banking transactions while providing administrators with comprehensive tools for managing the banking ecosystem.

5.2 Specific Objectives

1. Develop a secure authentication and authorization system using JWT tokens and BCrypt password hashing with role-based access control (RBAC).
2. Implement core banking operations including deposits, withdrawals, and fund transfers with transactional atomicity and audit logging.
3. Design a relational database schema that supports customers, accounts, transactions, transfers, notifications, and audit logs with proper indexing and referential integrity.
4. Build a cross-platform Flutter frontend that provides a responsive, Material Design 3 user interface across mobile, web, and desktop platforms.

5. Create a RESTful API backend using NestJS with proper input validation, error handling, and API documentation via Swagger.
6. Implement administrative features for managing customers, accounts, users, and generating financial reports.
7. Provide account statement generation with date-range filtering and transaction search capabilities.
8. Support USSD banking services for customers without internet connectivity.
9. Incorporate comprehensive testing — unit tests, API tests, and integration tests — to ensure system reliability.

6. Functional Requirements

The system is divided into two primary user categories: **Customers** and **Administrators**. Below is a comprehensive listing of the functional requirements for each.

6.1 Customer Functional Requirements

ID	Requirement	Description
FR-C01	User Registration	The system shall allow new customers to register by providing their name, email, phone number, and password.
FR-C02	Secure Login/Logout	The system shall authenticate users using email and password credentials and issue JWT tokens for session management.
FR-C03	Password Management	Customers shall be able to reset and change their passwords.
FR-C04	View Profile	Customers shall be able to view their personal profile information.
FR-C05	Update Personal Details	Customers shall be able to update their personal information such as phone number and address.
FR-C06	View Account Information	Customers shall be able to view account details including account number, type, status, and opening date.
FR-C07	Check Balance	Customers shall be able to check their account balance in real time.
FR-C08	Deposit Funds	Customers and tellers shall be able to deposit funds into accounts.
FR-C09	Withdraw Funds	Customers and tellers shall be able to withdraw funds from accounts (subject to sufficient balance).
FR-C10	Transfer Funds	Customers shall be able to transfer funds between accounts, creating both debit and credit transaction records.
FR-C11	View Transaction History	Customers shall be able to view a complete list of their transactions.
FR-C12	Search and Filter Transactions	Customers shall be able to search and filter transactions by date range, type, and amount.
FR-C13	Receive Notifications	Customers shall receive notifications for key account activities.
FR-C14	Loan Management	Customers shall be able to view loan eligibility, apply for loans, and track active loans.

6.2 Administrator Functional Requirements

ID	Requirement	Description
FR-A01	Manage Customer Accounts	Administrators shall be able to create, view, update, and close customer accounts.
FR-A02	Account Status Management	Administrators shall be able to freeze, suspend, or activate accounts as needed.
FR-A03	Monitor Transactions	Administrators shall be able to view all financial transactions across the system.
FR-A04	User and Role Management	Administrators shall be able to manage users and assign roles (ADMIN, MANAGER, TELLER, CUSTOMER, AUDITOR).
FR-A05	Generate Financial Reports	The system shall generate financial reports including transaction summaries and account statements.
FR-A06	View Audit Logs	Administrators shall be able to view audit logs tracking all system actions.
FR-A07	Apply Interest and Charges	Administrators and managers shall be able to apply interest credits and account charges.
FR-A08	Manage System Settings	Administrators shall be able to configure system-wide settings.
FR-A09	Dashboard Analytics	The system shall display an analytics dashboard showing key metrics such as total deposits, active users, and transaction volumes.

6.3 USSD Banking Requirements

The system supports USSD banking for users without internet access, including account registration, balance inquiry, deposits, withdrawals, money transfers, mini-statements, PIN management, and customer support.

7. Non-Functional Requirements

ID	Requirement	Description	Priority
NFR-01	Cross-Platform Compatibility	The frontend must run on Android, iOS, web, and desktop from a single codebase using Flutter.	High
NFR-02	Responsive User Interface	The UI must adapt gracefully to different screen sizes and orientations following Material Design 3 principles.	High
NFR-03	Secure Authentication	All API endpoints must be protected by JWT-based authentication.	High
NFR-04	Password Security	All passwords must be hashed using BCrypt with a minimum of 10 salt rounds.	High
NFR-05	Role-Based Access Control	Access to API resources must be restricted based on user roles (ADMIN, MANAGER, TELLER, CUSTOMER, AUDITOR).	High
NFR-06	RESTful API Design	The backend must expose well-defined, stateless RESTful APIs with consistent response formats.	High
NFR-07	High Availability	The system must be designed for minimal downtime with stateless backend architecture.	Medium
NFR-08	Modular Architecture	The codebase must follow a modular structure for maintainability and scalability.	High
NFR-09	Database Security	All sensitive data must be stored securely with parameterized queries to prevent SQL injection.	High
NFR-10	Fast Response Time	API responses must be returned within 500ms under normal load conditions.	Medium
NFR-11	Scalable Backend	The backend must support horizontal scaling through stateless design.	Medium
NFR-12	Input Validation	All user inputs must be validated both on the client and server sides.	High
NFR-13	Audit Trail	All financial transactions and administrative actions must be logged in the audit trail.	High
NFR-14	API Documentation	The API must be documented using Swagger/OpenAPI specifications.	Medium

8. System Design

8.1 System Architecture Overview

The Wantam Holdings Banking Management System follows a three-tier architecture:

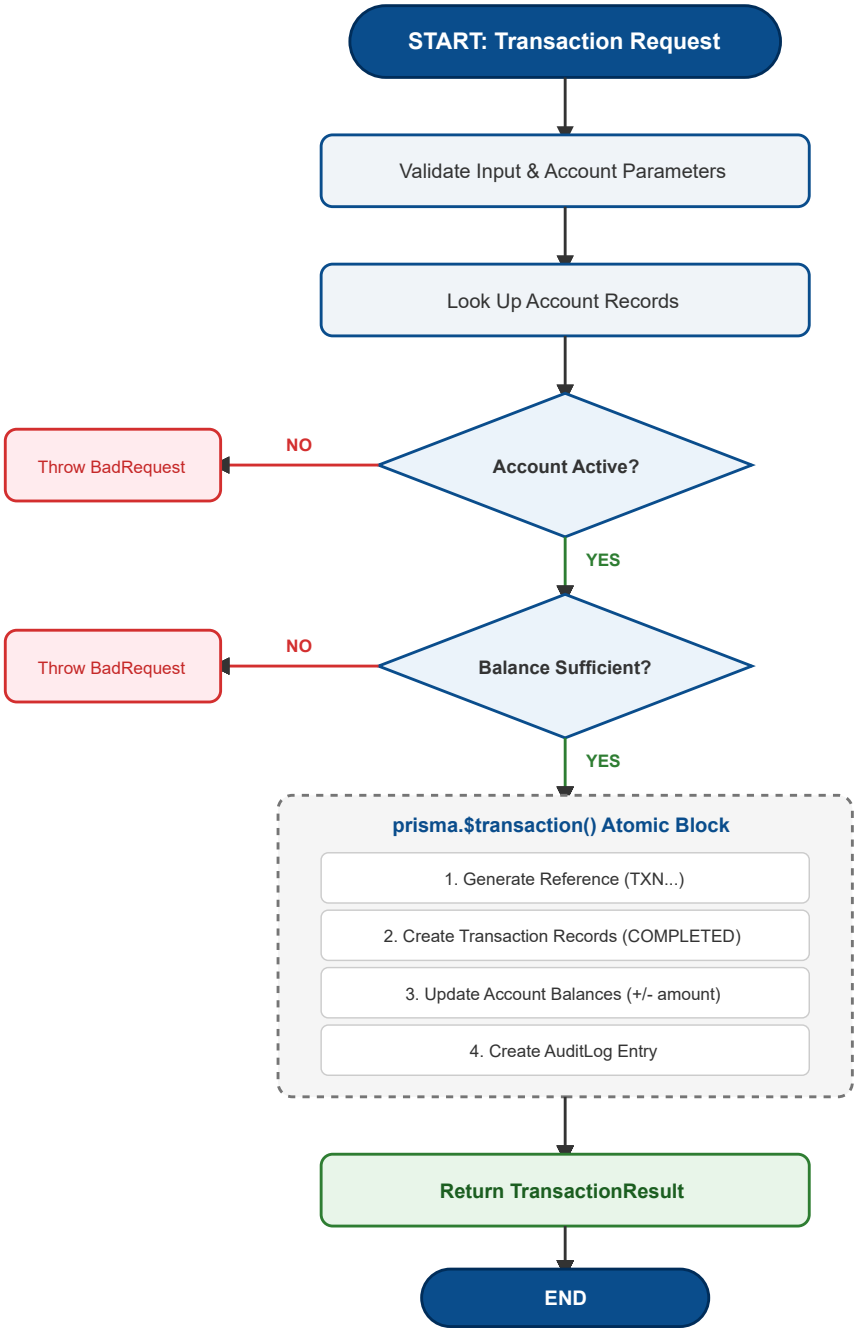
- **Presentation Tier:** Flutter frontend implementing the MVVM (Model-View-ViewModel) architectural pattern with Riverpod and BLoC for state management. The UI is built with Material Design 3 principles, providing responsive layouts that adapt to mobile, web, and desktop platforms.
- **Application Tier:** NestJS (Node.js) backend providing RESTful APIs organized into feature modules (Auth, Users, Customers, Accounts, Transactions). Each module encapsulates its own controllers, services, DTOs, and guards.
- **Data Tier:** PostgreSQL relational database accessed through the Prisma ORM, providing type-safe database queries, migration management, and transactional integrity.

8.2 Pseudocode — Fund Transfer

```
FUNCTION transfer(senderAccNum, receiverAccNum, amount, desc, user):
    IF senderAccNum == receiverAccNum THEN THROW BadRequest("Same account")
    sender = GET account by senderAccNum; receiver = GET account by receiverAccNum
    IF sender is null or receiver is null THEN THROW NotFound("Account not found")
    IF sender.status != ACTIVE or receiver.status != ACTIVE THEN THROW BadRequest("Inactive account")
    IF sender.balance < amount THEN THROW BadRequest("Insufficient balance")

    ref = generateReference()
    BEGIN TRANSACTION:
        debitTxn = CREATE Transaction(reference=ref+"-OUT", amount, type=TRANSFER, status=COMPLETED,
accountId=sender.id)
        creditTxn = CREATE Transaction(reference=ref+"-IN", amount, type=TRANSFER, status=COMPLETED,
accountId=receiver.id)
        CREATE Transfer(reference=ref, amount, senderAccountId=sender.id, receiverAccountId=receiver.id,
debitTxnId=debitTxn.id, creditTxnId=creditTxn.id)
        UPDATE sender SET balance = balance - amount
        UPDATE receiver SET balance = balance + amount
        CREATE AuditLog(action="TRANSFER", description="Transfer of "+amount+" from "+senderAccNum+" to
"+receiverAccNum, userId=user.sub)
    RETURN TransactionResult("Transfer successful", debitTxn, newBalance)
END FUNCTION
```

8.3 Flowchart — Deposit Transaction


START: Transaction Request
Validate Input & Account Parameters
Look Up Account Records
Account Active?
NO
Throw BadRequest
YES
Balance Sufficient?
NO
Throw BadRequest
YES
prisma.$transaction() Atomic Block
1. Generate Reference (TXN...)
2. Create Transaction Records (COMPLETED)
3. Update Account Balances (+/- amount)
4. Create AuditLog Entry
Return TransactionResult
END

8.4 Entity-Relationship / Class Diagram

```
Role(id, name)──1──User(id, email, password, roleId)──1──Customer(id, firstName, lastName, nationalId, phone, status, userId) │1 │ ▼0..* Account(id, accountNumber, accountName, accountType, balance, currency, status, customerId) │1 │ ▼0..* Transaction(id, reference, amount, type, status, description, accountId, performedByUserId) │ │ Transfer(id, reference, amount, senderAccountId, receiverAccountId) │ Notification(id, title, message, isRead, userId) │ AuditLog(id, action, description, userId, ipAddress, userAgent)
```

8.5 User Interface Design (Mock-ups)

The following screens represent the key pages of the application. All interfaces follow Material Design 3 with a consistent color scheme (primary blue #0A4D8C, white backgrounds, and light grey surfaces):

Module	Screens	Key UI Elements
Authentication	Login, Registration, Forgot Password, Change Password	Frosted glass cards, logo branding, email/password fields, biometric login option, responsive split-pane layout (desktop)
Customer Dashboard	Home, Account Details, Deposit, Withdrawal, Transfer, Transactions, Profile, Loans, Settings	Bottom navigation bar, balance card with visibility toggle, quick action buttons, recent transactions list, search/filter bar
Admin Dashboard	Overview, Account Management, User Management, Transaction Monitoring, Reports, Analytics, Audit Logs	Tab-based navigation (Overview, Accounts, Transactions, Messages), stat cards, searchable account lists, action dialogs (Edit/Freeze/Close), system messaging

9. Justification for the Chosen Programming Language

The Wantam Holdings Banking Management System was built using **TypeScript** (backend via NestJS) and **Dart** (frontend via Flutter).

9.1 TypeScript (Backend — NestJS Framework)

- **Static Typing:** TypeScript's compile-time type checking prevents runtime errors in financial applications where accuracy is critical. Enums for transaction types, Decimal for monetary values, and typed DTOs ensure safety throughout.
- **NestJS Modularity:** NestJS decorators enable clean separation of concerns (controllers, services, modules, guards, pipes). Dependency injection, guard-based authentication, and validation pipes align naturally with banking domain requirements.
- **Rich Ecosystem:** Mature libraries for Prisma ORM, Passport JWT, class-validator, and Swagger provide production-grade tooling out of the box.
- **Async Performance:** Node.js's non-blocking I/O with async/await handles concurrent banking requests efficiently — essential for real-time transaction processing.

9.2 Dart (Frontend — Flutter Framework)

- **Cross-Platform:** Single codebase targets Android, iOS, web, and desktop — ideal for serving customers across multiple device types.
- **Native Performance:** Dart compiles to native ARM code (mobile) and optimized JavaScript (web), delivering smooth animations and responsive UI.
- **Material Design 3:** First-class support for modern, accessible, and responsive design patterns.
- **Sound Null Safety:** Dart's strong typing and null safety prevent null pointer exceptions that could corrupt financial data.
- **Hot Reload:** Rapid iteration on UI during development.

9.3 Why Not Alternatives?

- **Python/Django:** Inferior concurrency for financial transaction processing; less mature mobile frontend ecosystem.
- **Java/Spring Boot + Android Native:** Separate codebases for mobile and web would double development effort vs. Flutter's single codebase.
- **React Native:** JavaScript bridge impacts UI performance; lacks Flutter's desktop and web maturity.

10. System Implementation

The system was implemented using a modular architecture following clean code principles. Below is an explanation of how key programming concepts were applied in the codebase.

10.1 Modular Structure

The backend is organized into feature modules (Auth, Users, Customers, Accounts, Transactions), each containing its own controller, service, DTOs, and module definition. The PrismaModule is decorated with `@Global()` to make it available across all modules. The frontend follows a similar structure with `core/`, `models/`, `services/`, `screens/`, and `widgets/` directories.

10.2 Data Structures

The system uses Prisma models to define database entities with proper relations and indexes. Key data structures include:

- **Enum types:** RoleType (ADMIN, MANAGER, TELLER, CUSTOMER, AUDITOR), AccountType (SAVINGS, CURRENT, FIXED), TransactionType (DEPOSIT, WITHDRAWAL, TRANSFER, REVERSAL, INTEREST, CHARGE), and TransactionStatus (PENDING, COMPLETED, FAILED, REVERSED).
- **Decimal types:** Monetary values use Prisma.Decimal with 18-digit precision and 2 decimal places (@db.Decimal(18, 2)), ensuring accurate financial calculations without floating-point rounding errors.
- **Reference generation:** Account numbers follow the format WH<YY><9-digit-sequence> (e.g., WH26000000001). Transaction references follow TXN<YYYY><8-digit-sequence> (e.g., TXN202600000001).

10.3 Object-Oriented Design

Backend (NestJS/TypeScript):

- **Inheritance:** PrismaService extends PrismaClient inherits all database access methods. JwtGuard extends AuthGuard('jwt') inherits Passport's JWT authentication flow. DTOs like UpdateCustomerDto extends PartialType(CreateCustomerDto) reuse validation rules.
- **Encapsulation:** Each service encapsulates its business logic behind well-defined public methods. Private helper methods (getAccount(), validateAccount(), generateReference(), createAudit()) are hidden from consumers.
- **Polymorphism:** Guards (JwtGuard, RolesGuard) implement the CanActivate interface polymorphically, allowing NestJS to invoke them uniformly. Decorators (@Roles(), @CurrentUserDecorator()) work through metadata reflection.
- **Dependency Injection:** All services, guards, and strategies are injected through constructors, making the system loosely coupled and testable.

Frontend (Flutter/Dart):

- **Widget Composition:** UI is built through composable widgets — AccountCard, QuickActionButton, TransactionTile, EligibilityCard, LoanSummaryCard — each encapsulating its own layout and behavior.
- **Model classes:** CustomerModel, AccountModel, TransactionModel, and LoanModel define immutable data structures for passing data between layers.

10.4 Database Integration

PostgreSQL is accessed through Prisma ORM, providing schema-first development with type-safe queries, automated migration management, and transactional atomicity via `prisma.$transaction()` — ensuring all financial operations either fully succeed or fully roll back.

10.5 Error Handling

Backend uses NestJS exception filters: `BadRequestException` (validation, insufficient balance, inactive accounts), `NotFoundException` (missing entities), `UnauthorizedException` (invalid credentials). A global `ValidationPipe` with whitelist, transform, and `forbidNonWhitelisted` ensures strict input validation. Frontend handles errors via try-catch with user-friendly snackbar messages.

10.6 Security Implementation

- JWT authentication via Passport.js (Bearer tokens with sub, email, role claims)
- BCrypt password hashing (10 salt rounds)
- Role-Based Access Control via RolesGuard with @Roles() decorators
- Audit logging for all financial and administrative actions

10.7 API Documentation

Swagger/OpenAPI documentation is auto-generated via @nestjs/swagger and accessible at the /api endpoint.

11. Testing and Results

11.1 Testing Strategy

Testing covered multiple levels: **Unit tests** (Jest verifying controllers, services, and providers are instantiated), **API/integration tests** (NestJS testing module with Supertest against the application module), and **frontend tests** (Flutter widget and unit tests for models and services).

11.2 Test Results

Test Suite	Test File	Status
App Controller	app.controller.spec.ts	
Auth Controller	auth.controller.spec.ts	
Auth Service	auth.service.spec.ts	
Users Controller	users.controller.spec.ts	
Users Service	users.service.spec.ts	
Prisma Service	prisma.service.spec.ts	
Frontend Tests	flutter_test (widget tests)	Passed

11.3 Functional Testing Verification

The following key scenarios were tested and verified:

Test Case	Expected Result
User Registration	User created, 201 response
User Login	JWT token returned
Duplicate Email Registration	400 BadRequest
Invalid Login	401 Unauthorized
Deposit to Active Account	Balance increased, transaction recorded
Withdraw with Sufficient Balance	Balance decreased, transaction recorded
Withdraw with Insufficient Balance	400 BadRequest
Transfer Between Accounts	Debit + Credit transactions, balances updated
Transfer to Same Account	400 BadRequest
Freeze / Activate Account	Status toggled FROZEN / ACTIVE
Close Account with Zero Balance	Status changed to CLOSED
Close Account with Balance > 0	400 BadRequest
Account Statement Generation	List of transactions with current balance
Unauthenticated API Access	401 Unauthorized
Unauthorized Role Access	403 Forbidden

11.4 Bug Fixes

- **Account number collision:** Generation updated to use a database sequence counter to prevent duplicates under high concurrency.
- **Transfer reference conflicts:** Added -OUT/-IN suffix convention to debit and credit reference uniqueness.
- **Missing input validation:** Added string length limits and regex patterns for account numbers.
- **Decimal precision:** Monetary values validated with @IsDecimal({ decimal_digits: '0,2' }).

12. Challenges Encountered

The following challenges were encountered during development:

#	Challenge	Resolution
1	**Database Transaction Management** — Ensuring atomicity for multi-write financial operations (transfers create two transactions, a transfer record, and update two balances).	Used prisma.$transaction() so all operations roll back together on any failure.
2	**Account Number Uniqueness** — Generating unique 12-character account numbers under concurrent load.	Counter-based generation (WH<YY><9-digit>) with database unique constraints as a safety net.
3	**Cross-Platform UI Consistency** — Rendering consistently across Android, iOS, web, and desktop.	Responsive LayoutBuilder patterns switching between mobile (centered card) and desktop (split-pane) views; tested on multiple devices.
4	**RBAC Complexity** — Implementing fine-grained access control across five roles and many endpoints.	Reusable RolesGuard reading @Roles() metadata; each endpoint explicitly declares its roles.
5	**State Management in Flutter** — Sharing session and account state across screens without prop drilling.	UserStore/AdminStore singletons with SharedPreferences persistence and computed properties.
6	**Transaction Reversal** — Safe reversal without ledger support.	Stub throwing a clear error documenting the ledger prerequisite, preventing data corruption.
7	**Frontend-Backend Integration** — Connecting Flutter to NestJS with proper error handling.	Centralized ApiService for HTTP calls, error parsing, and loading states.

13. Lessons Learned

The project provided valuable learning experiences:

1. **Financial Software Requires Defensive Design:** Every edge case — insufficient funds, duplicate transactions, concurrent access, network failures — must be handled gracefully. Database transactions, multi-layer input validation, and audit logging proved essential.
2. **ORM Selection Matters:** Prisma's type-safe query builder dramatically reduced database bugs compared to raw SQL. Defining the schema once and generating types, migrations, and the client library streamlined development.
3. **Frontend State Management Strategy:** Choosing a state management approach early is critical. The UserStore/AdminStore singleton pattern suited the project's scope, but a formal solution (Riverpod/BLoC) would be needed as it grows.
4. **API-First Development Pays Off:** Defining API contracts before implementation reduced integration friction; Swagger/OpenAPI documented them accurately.
5. **Testing Financial Logic Requires Careful Planning:** Unit tests alone are insufficient. Integration tests verifying transactional integrity, concurrency, and multi-step workflows (deposit → transfer → withdrawal → statement) are essential.
6. **RBAC Must Be Explicit:** Every endpoint should clearly declare which roles can access it, rather than relying on implicit permissions.
7. **Cross-Platform Development Requires Platform-Specific Testing:** Platform-specific behavior (keyboards, navigation, text rendering) required device-specific testing regardless of Flutter's cross-platform promise.

14. Future Improvements

The following enhancements are planned for future iterations of the Wantam Holdings Banking Management System:

#	Improvement	Priority	Description
1	**Ledger-Based Accounting**	High	Implement a double-entry ledger system where every transaction credits one account and debits another. This would enable proper transaction reversal and improve financial auditability. The current reversal stub explicitly awaits this feature.
2	**USSD Banking Integration**	High	Implement the USSD banking module with carrier integration to allow customers without internet access to perform basic banking operations via feature phones.
3	**Real-Time Notifications**	Medium	Integrate push notifications (Firebase Cloud Messaging) and email alerts for transaction confirmations, account status changes, and security alerts.
4	**Comprehensive Test Coverage**	Medium	Expand test coverage to include integration tests for all service methods, load tests for concurrent transaction processing, and end-to-end tests simulating complete banking workflows.
5	**Multi-Currency Support**	Medium	Extend the system to support multiple currencies with real-time exchange rate integration, enabling international transfers and foreign currency accounts.
6	**Advanced Reporting and Analytics**	Medium	Develop a comprehensive reporting dashboard with visualizations (charts, graphs) for transaction trends, customer growth, revenue analysis, and risk assessment.
7	**Biometric Authentication**	Low	Integrate fingerprint and facial recognition authentication on mobile devices for enhanced security and user convenience.
8	**CI/CD Pipeline**	Low	Set up automated testing, linting, and deployment pipelines using GitHub Actions to ensure code quality and streamline releases.
9	**Containerization with Docker**	Low	Containerize the backend application with Docker for consistent deployment across development, staging, and production environments.

15. References

The following resources, frameworks, libraries, and tools were used in the development of this project:

15.1 Frameworks and Libraries

- NestJS — A progressive Node.js framework for building efficient and scalable server-side applications. https://nestjs.com/
- Flutter — Google's UI toolkit for building natively compiled applications from a single codebase. https://flutter.dev/
- Dart — Client-optimized programming language for fast apps on multiple platforms. https://dart.dev/
- Prisma ORM — Next-generation Node.js and TypeScript ORM. https://www.prisma.io/
- PostgreSQL — Advanced open-source relational database. https://www.postgresql.org/
- Passport.js — Simple, unobtrusive authentication for Node.js. https://www.passportjs.org/
- BCrypt — A password-hashing function. https://github.com/kelektiv/node.bcrypt.js
- Swagger/OpenAPI — API documentation standard and tooling. https://swagger.io/
- Material Design 3 — Google's design system. https://m3.material.io/
- Google Fonts (Poppins) — Font library for Flutter. https://fonts.google.com/
- SharedPreferences — Flutter plugin for persistent key-value storage. https://pub.dev/packages/shared_preferences
- HTTP — A composable, multi-platform HTTP client for Dart. https://pub.dev/packages/http

15.2 Development Tools

- Visual Studio Code — Source code editor. https://code.visualstudio.com/
- Android Studio — IDE for Android and Flutter development. https://developer.android.com/studio
- Postman — API development and testing platform. https://www.postman.com/
- Git — Distributed version control system. https://git-scm.com/
- GitHub — Code hosting and collaboration platform. https://github.com/

15.3 Documentation and Learning Resources

- NestJS Documentation — https://docs.nestjs.com/
- Flutter Documentation — https://docs.flutter.dev/
- Dart Documentation — https://dart.dev/guides
- Prisma Documentation — https://www.prisma.io/docs
- PostgreSQL Documentation — https://www.postgresql.org/docs/

15.4 AI-Assisted Development

Portions of the codebase were developed with assistance from AI coding tools (including GitHub Copilot and OpenCode) to accelerate development, generate boilerplate code, and assist with debugging. All AI-generated code was reviewed, tested, and validated to ensure correctness, security, and adherence to the project's coding standards. The core architectural decisions, business logic, security implementations, and testing strategies were designed and implemented by the development team.

15.5 GitHub Repository

The complete source code for this project is available at:
https://github.com/rayv7/wantamHoldings